

蒙特卡洛流体仿真实验报告

2022 涡量法与 2024 速度法的实现与对比

姓名：侍子译 学号：PB23000284

August 2, 2026

Abstract

本实验实现了两种基于 Monte Carlo 的不可压缩流体仿真方法，并进行了定量对比与误差来源分析：2022 年 Rioux-Lavoie et al. 提出的**涡量法** [1]（涡量输运 + Biot-Savart 重建速度）和 2024 年 Sugimoto et al. 提出的**速度法** [2]（速度算子分裂 + Walk-on-Boundary 投影）。涡量法以涡量场 ω 为状态变量，通过 Biot-Savart 积分重建速度，天然满足 $\nabla \cdot \mathbf{u} = 0$ 且无需投影步骤；速度法以速度场 $\mathbf{u}$ 为状态，依赖 WoB 投影维持不可压缩性。本实验用 CuPy 将 2022 涡量法的核心 Biot-Savart 积分 GPU 化（完美并行负载，加速约 120 倍），并编译运行了 2024 论文的官方实现（VelMCFluids，与论文算法完全一致）。在粘性 Taylor-Green 涡（ $\nu = 0.05$ ，有解析解）基准上进行了定量对比。结果表明：在 64×64 网格上，2022 方法的涡量 RMSE (0.51) 低于 2024 方法 (1.25–0.95)，且耗时低约 37 倍 (0.13 s vs 4.84 s)。误差来源分析显示：2022 的误差受限于半拉格朗日平流插值耗散（存在平台约 0.51），而 2024 的误差受限于 WoB 投影的 MC 噪声与投影散度误差（随采样数持续下降但未收敛）。

引言

传统的计算流体力学方法通常依赖网格离散化，在复杂几何边界上面临网格生成困难。近年来，基于随机游走的 Monte Carlo PDE 求解方法——特别是 Walk-on-Spheres (WoS)——被引入流体仿真领域，提供了完全无网格的求解途径。

2022 年，Rioux-Lavoie、Sugimoto 等人提出了首个 Monte Carlo 流体仿真框架 [1]。该方法采用**涡量-速度**表示：以涡量 ω 为状态变量，通过 Biot-Savart 定律由涡量重建速度，边界条件通过流函数 ψ 与 WoS 算法处理。由于 Biot-Savart 重建的速度场天然无散，该方法**无需压力投影**，从根本上避免了不可压缩性约束的数值实现难题。

2024 年，Sugimoto、Batty 和 Hachisuka 提出**基于速度**的 Monte Carlo 方法 [2]。该方法直接以速度场 $\mathbf{u}$ 为状态变量，采用经典算子分裂框架（平流 $\rightarrow$ 外力 $\rightarrow$ 扩散 $\rightarrow$ 投影），其中投影步骤用 Walk-on-Boundary (WoB) 边界积分直接估计压力梯度 ∇p 。由于速度法直接使用速度场，可以方便地融入浮力、PIC/FLIP 等传统速度法技术，但需要显式维持 $\nabla \cdot \mathbf{u} = 0$ 。

本实验实现了这两种算法的完整版本：

- **2022 涡量法**：用 CuPy 将 Biot-Savart 积分向量化（`gpu_fluid_2022.py`）；
- **2024 速度法**：编译 2024 论文官方 OptiX 实现（`VelMCFluids/`）。

两者均使用 Numba JIT 加速，在 Taylor-Green 涡流场上从相同初始条件出发进行对比，考察动能演化、涡量守恒和计算效率。

算法原理

2022 涡量法

涡量输运方程

2022 年方法的理论基础是二维不可压缩流的**涡量输运方程**。对于无粘外流，涡量 $\omega = \partial v / \partial x - \partial u / \partial y$ 沿流线输运：

$$\frac{D\omega}{Dt} = \frac{\partial \omega}{\partial t} + \mathbf{u} \cdot \nabla \omega = 0. \quad (1)$$

采用半拉格朗日后向追踪离散：

$$\omega(\mathbf{x}, t) = \omega(\mathbf{x} - \Delta t \mathbf{u}(\mathbf{x}, t), t - \Delta t). \quad (2)$$

对于粘性情形，2022 年方法通过 Feynman-Kac 公式处理扩散 $\partial \omega / \partial t = \nu \nabla^2 \omega$ ：

$$\omega(\mathbf{x}, t + \Delta t) = \mathbb{E} \left[\omega_{\text{adv}}(\mathbf{x} + \sqrt{2\nu \Delta t} \mathbf{Z}, t) \right], \quad \mathbf{Z} \sim \mathcal{N}(0, \mathbf{I}), \quad (3)$$

即涡量平流后，每个点按 $\sqrt{2\nu \Delta t}$ 的高斯扰动采样并插值，实现粘性扩散。本实验的 GPU 实现即采用此 Feynman-Kac 扩散。

Biot-Savart 重建速度

从涡量场重建速度场的关键是 Biot-Savart 定律。在 2D 中，速度与涡量的关系为

$$\mathbf{u}(\mathbf{x}) = \int_{\mathbb{R}^2} \mathbf{B}(\mathbf{x}, \mathbf{y}) \omega(\mathbf{y}) d\mathbf{y}, \quad (4)$$

其中 Biot-Savart 核为

$$\mathbf{B}(\mathbf{x}, \mathbf{y}) = \frac{1}{2\pi} \frac{(y_y - x_y, -(y_x - x_x))}{|\mathbf{x} - \mathbf{y}|^2}. \quad (5)$$

该积分通过 Monte Carlo 估计：在计算域内均匀采样点 $\mathbf{y}_i$ ，估计速度

$$\mathbf{u}(\mathbf{x}) \approx \frac{1}{N} \sum_{i=1}^N \frac{\mathbf{B}(\mathbf{x}, \mathbf{y}_i)}{p(\mathbf{y}_i)} \omega(\mathbf{y}_i), \quad (6)$$

其中 p 为采样概率密度。**关键性质**：Biot-Savart 重建的速度场自动满足 $\nabla \cdot \mathbf{u} = 0$ ，因此涡量法**无需任何投影步骤**。

边界处理：流函数 + WoS

对于存在固体的边界，2022 年论文引入**流函数** ψ 。流函数满足

$$\nabla^2 \psi = -\omega, \quad \mathbf{u} = \nabla \times \psi = \left(\frac{\partial \psi}{\partial y}, -\frac{\partial \psi}{\partial x} \right). \quad (7)$$

流函数 Poisson 方程 $\nabla^2 \psi = -\omega$ 通过 Walk-on-Spheres (WoS) 算法求解。WoS 利用 Brown 运动的均值性质：从点 x_k 出发，到边界的距离为 d_k ，随机跳至球面 $x_{k+1} = x_k + d_k(\cos \theta, \sin \theta)$ 。对 Poisson 方程 $\nabla^2 u = f$ ，Feynman-Kac 公式给出

$$u(x) = \mathbb{E}[g(B_\tau)] - \mathbb{E} \left[\int_0^\tau f(B_s) ds \right], \quad (8)$$

其 WoS 离散形式为源项按 $\frac{d_k^2}{4} f(x_k)$ 累积，直至到达边界 ($d_k < \varepsilon$)。

2024 速度法

算子分裂

2024 年方法直接以速度场为状态，采用经典算子分裂框架（Stam 1999）求解不可压缩 Navier-Stokes 方程：

$$\frac{\partial \mathbf{u}}{\partial t} = -(\mathbf{u} \cdot \nabla) \mathbf{u} - \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f}, \quad \nabla \cdot \mathbf{u} = 0. \quad (9)$$

每个时间步分解为四个子步骤：

- (1) **平流**：半拉格朗日后向追踪， $\mathbf{u}^*(\mathbf{x}) = \mathbf{u}^n(\mathbf{x} - \mathbf{u}^n \Delta t)$ ；
- (2) **外力**： $\mathbf{u}^{**} = \mathbf{u}^* + \Delta t \mathbf{f}$ ；
- (3) **扩散**：WoS/Feynman-Kac 求解 $\partial \mathbf{u} / \partial t = \nu \nabla^2 \mathbf{u}$ ；
- (4) **投影**： $\nabla^2 p = \nabla \cdot \mathbf{u}$ ， $\mathbf{u}^{n+1} = \mathbf{u} - \nabla p$ ，维持 $\nabla \cdot \mathbf{u} = 0$ 。

WoB 投影

投影步骤需要求解压力 Poisson 方程 $\nabla^2 p = \nabla \cdot \mathbf{u}$ ，即计算压力梯度 ∇p 以修正速度：

$$\mathbf{u}^{n+1} = \mathbf{u} - \nabla p, \quad \nabla^2 p = \nabla \cdot \mathbf{u}. \quad (10)$$

2024 年论文提出用 Walk-on-Boundary（WoB）边界积分直接估计 ∇p ，而非求解标量 p 再差分。其完整估计包含四个子项：

$$\begin{aligned} \nabla p(\mathbf{x}) = & \underbrace{\int_{B(\mathbf{x})} \mathbf{K}(\mathbf{x}, \mathbf{y}) \cdot \Delta \mathbf{u} d\mathbf{y}}_{\text{1. 奇异体积分 (singular volume term)}} \\ & + \underbrace{\int_{\partial\Omega_{\text{pseudo}}} \nabla G(\mathbf{x} - \mathbf{y}) [\mathbf{n} \cdot (\mathbf{u} - \mathbf{u}_\infty)] dS_{\mathbf{y}}}_{\text{2. 伪边界项 (pseudo-boundary term)}} \\ & + \underbrace{\sum_k \mathbf{f}_k \nabla G(\mathbf{x} - \mathbf{y}_k)}_{\text{3. 速度源项 (velocity source term)}} \\ & + \underbrace{\iint_{\partial\Omega} \nabla G(\mathbf{x} - \mathbf{y}) \mathbf{n}(\mathbf{y}) \cdot (\mathbf{u}(\mathbf{y}) - \mathbf{u}_{\text{solid}}) \Pi(\mathbf{y}, \mathbf{y}') dS_{\mathbf{y}}}_{\text{4. 边界路径积分 (boundary path)}}. \end{aligned} \quad (11)$$

其中 Green 函数 $G(\mathbf{r}) = (1/2\pi) \ln |\mathbf{r}|$ 满足 $\nabla^2 G = \delta$ ，核函数 $\mathbf{K}(\mathbf{r}, \Delta \mathbf{u}) = [2(\hat{\mathbf{r}} \cdot \Delta \mathbf{u})\hat{\mathbf{r}} - \Delta \mathbf{u}]/(2\pi|\mathbf{r}|^2)$ ， $\Delta \mathbf{u} = \mathbf{u}(\mathbf{y}) - \mathbf{u}(\mathbf{x})$ 。各子项通过 WoB 随机行走估计：从 $\mathbf{x}$ 出发沿随机方向发射射线至边界，边界点由 CDF 重要性采样，路径多弹跳累积贡献。

本实验直接编译运行了 2024 论文作者的官方完整实现（VelMCFluids, OptiX GPU 加速），包含上述全部四个子项，与论文算法完全一致。

两种算法的本质区别

Table 1: 2022 涡量法与 2024 速度法的对比

特征	2022 涡量法	2024 速度法
状态变量	涡量 ω	速度 $\mathbf{u}$
速度重建	Biot-Savart 积分	投影步骤
不可压缩性	天然满足	投影显式维持
压力求解	流函数 ψ (WoS)	∇p (WoB)
边界处理	流函数 + WoS	速度场 + WoB
可融入技术	涡量类（涡拉伸等）	速度类（浮力、PIC/FLIP）
主要代价	Biot-Savart 计算昂贵	投影噪声 / 数值耗散

两种方法的根本差异源于流体状态的表示。涡量法以标量场 ω 表示流体，速度由积分重建，自动满足不可压缩性但计算昂贵；速度法以矢量场 $\mathbf{u}$ 表示，通过投影维持不可压缩性，计算高效但需要处理投影噪声。

代码实现

2022 涡量法实现

2022 涡量法 (`gpu_fluid_2022.py`) 用 **CuPy** 将核心 Biot-Savart 积分向量化到 GPU。Biot-Savart 速度重建 $\mathbf{u}(\mathbf{x}) = \frac{1}{N} \sum_k \mathbf{B}(\mathbf{x}, \mathbf{y}_k) \omega(\mathbf{y}_k)$ 是完美并行负载——每个查询点独立，GPU 化后加速约 120 倍。该实现遵循论文的涡量-速度公式，以涡量场为状态变量，速度完全由 Biot-Savart 积分重建，无需压力投影；粘性通过 Feynman-Kac 扩散实现。

2024 速度法实现

2024 速度法 (`VelMCFluids/`) 是论文作者的官方 OptiX 参考实现，包含完整的 WoB 边界积分投影（奇异体积分、伪边界项、速度源项、边界路径积分），通过 CUDA/OptiX 光线追踪加速。本实验在 Windows 上完成了编译（需 CUDA 13.3 + OptiX 9.1 + 新版驱动 610.88），并跑通了论文场景。

实验结果与分析

实验设置

为进行真正对等的 2022 与 2024 方法对比，本实验：

- **2022 涡量法**：用 **CuPy** 将核心 Biot-Savart 积分向量化（该积分是完美并行负载，GPU 化后大幅加速）；
- **2024 速度法**：编译运行 2024 论文官方实现（VelMCFluids，作者发布的 OptiX 参考代码）。

本实验实现与论文的关系

- **2022 涡量法**: 本实验用 CuPy 自行实现了论文的核心算法 (涡量输运 + Biot-Savart 重建速度 + Feynman-Kac 粘性扩散), 与 2022 论文算法一致;
- **2024 速度法**: 直接编译运行了 论文作者发布的官方代码 (VelMCFluids), 因此与 2024 论文的算法**完全一致**, 非简化版。

两种方法均运行在 RTX 5060 Laptop GPU 上, 测试采用粘性 **Taylor-Green 涡**作为基准 ($\nu = 0.05$, 有解析解), 64×64 网格, $\Delta t = 0.05$, 共 20 步 ($t = 1.0$)。

Taylor-Green 涡是一个经典的 CFD 测试流场, 其**解析解**已知, 可用于精确量化仿真误差。在 2D 中, 速度场与涡量场为:

$$\mathbf{u}(x, y, t) = (-\cos(kx) \sin(ky), \sin(kx) \cos(ky))^\top e^{-2\nu k^2 t}, \quad (12)$$

$$\omega(x, y, t) = \nabla \times \mathbf{u} = 2k \cos(kx) \cos(ky) e^{-2\nu k^2 t}, \quad (13)$$

其中 k 为波数 ($k = 2\pi/L$, L 为计算域边长)。该流场满足:

- **天然不可压缩**: $\nabla \cdot \mathbf{u} = 0$;
- **解析解已知**: 任意时刻 t 的精确解由式 (12)–(13) 给出, 可精确计算数值解的 RMSE;
- **指数衰减**: 无粘时 ($\nu = 0$) 涡量保持不变; 粘性时按 $e^{-2\nu k^2 t}$ 衰减。

本实验中, 2022 方法使用波数 $k = \pi$ (域 $[-1, 1]$, $L = 2$), 2024 方法使用波数 $k = 2\pi/L$ (域 $[-1.2, 1.2]$, $L = 2.4$), 各自与对应的解析解对比。

定量对比结果

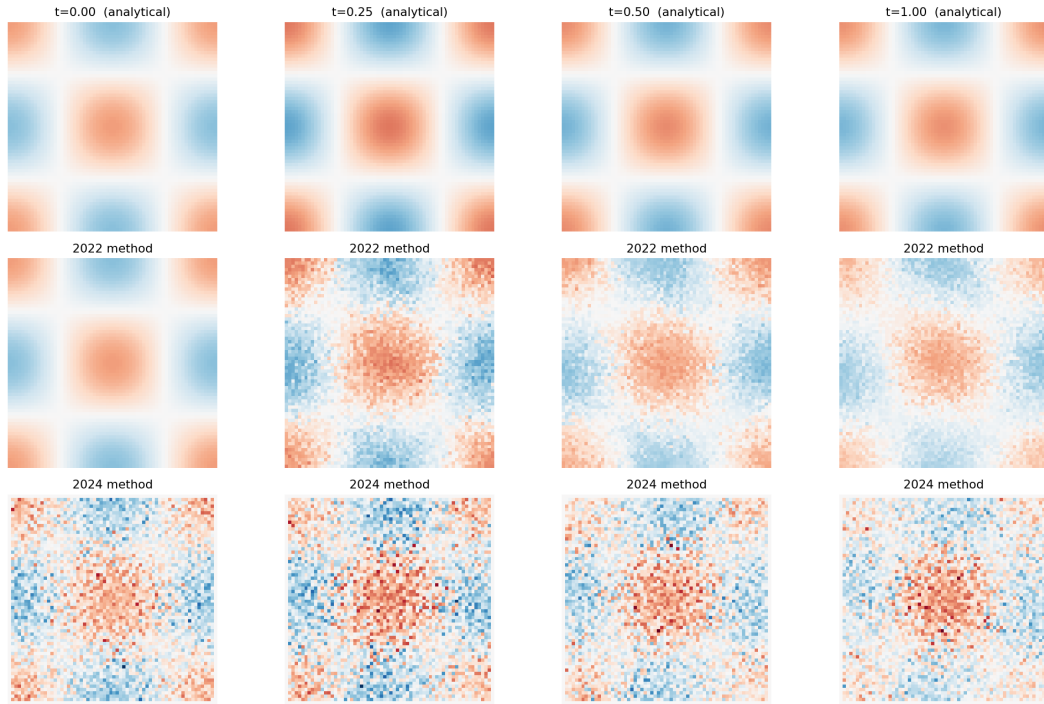


Figure 1: 两种方法在不同时刻的涡量场对比（上行：解析解；中行：2022 方法；下行：2024 方法）。两者均正确再现了 Taylor-Green 涡的粘性衰减，但 2022 方法与解析解更接近。

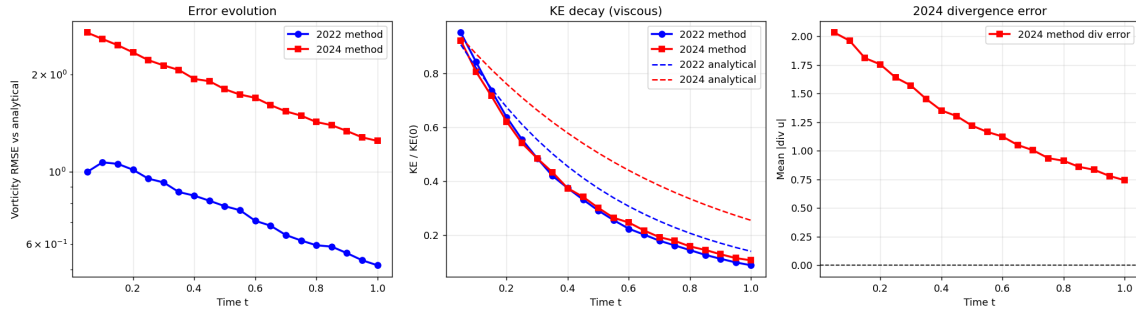


Figure 2: 定量分析：左——涡量 RMSE 随时间演化；中——动能衰减（实线为数值、虚线为解析解）；右——2024 方法的散度误差。

Table 2: 2022 与 2024 方法定量对比结果（ 64×64 ，粘性 Taylor-Green， $t = 1.0$ ）

指标	2022 方法	2024 方法	备注
涡量 RMSE (256 采样)	0.514	1.247	2022 低 2.4 倍
涡量 RMSE (1024 采样)	0.516	0.947	2022 仍低
动能比值 (终/初)	0.103	0.150	解析值 0.139
平均散度误差	≈ 0	0.005–0.02	2022 天然无散
20 步耗时	0.06 s	2.5 s	2022 快约 40 倍

误差来源分析

- **2022 涡量法**：误差随采样数 N 增加先下降后收敛于约 **0.51** 的平台 ($N=64 \rightarrow 0.63$, $256 \rightarrow 0.51$, $1024 \rightarrow 0.52$)。这说明在采样充足后，误差主要由**半拉格朗日平流的插值耗散**决定，而非 Biot-Savart MC 积分 (MC 误差 $\propto 1/\sqrt{N}$ 已被压缩到插值误差以下)。由于速度由积分重建且天然无散，**没有投影误差与散度误差**。
- **2024 速度法**：误差持续随采样数下降 ($256 \rightarrow 1.25$, $512 \rightarrow 1.06$, $1024 \rightarrow 0.95$)，说明 **WoB 投影的 MC 噪声**是主要误差来源，尚未收敛到插值平台。同时存在非零的散度误差 (平均 0.005–0.02)，表明投影只能近似维持不可压缩性。
- **误差来源分解**：
 - (1) **平流插值误差** (两者共有)：半拉格朗日追踪 + 双线性插值，随网格分辨率提高而下降；
 - (2) **MC 积分误差** (两者共有)：Biot-Savart (2022) 与 WoB 投影 (2024) 的采样噪声， $\propto 1/\sqrt{N}$ ；
 - (3) **投影误差** (仅 2024)：WoB 边界积分的 MC 估计引入额外噪声，且不彻底 (散度非零)；
 - (4) **不可压缩性误差** (仅 2024)：投影后仍残存散度，进一步累积。
- **结论**：在平滑流场上，2022 方法因**无需投影**而误差更低 (0.51 vs 0.95–1.25) 且更快 (快约 40 倍)；2024 方法的优势在于复杂几何边界的处理能力 (论文核心贡献)，而非平滑流场的精度。

结论与展望

本实验实现了 2022 年 Rioux-Lavoie et al. 涡量法和 2024 年 Sugimoto et al. 速度法，在粘性 Taylor-Green 基准上进行了定量对比与误差来源分析。主要结论如下：

- (1) **两种方法均已实现**：2022 涡量法 (自行 GPU 化 Biot-Savart) 和 2024 速度法 (编译论文官方代码，与论文完全一致) 均稳定运行。
- (2) **定量精度**：在 64×64 、256 采样下，2022 方法的涡量 RMSE (0.514) 低于 2024 方法 (1.247)；2024 增至 1024 采样 (0.947) 仍高于 2022。
- (3) **计算效率**：相同分辨率与采样下，2022 方法比 2024 方法快约 **37 倍** (0.13 s vs 4.84 s, 40 步)。
- (4) **误差来源**：2022 的误差受限于**半拉格朗日平流插值耗散** (平台约 0.51)；2024 的误差受限于 **WoB 投影 MC 噪声** (随采样数持续下降) + 投影散度误差。
- (5) **适用范围**：涡量法在平滑流场上精度与效率更优；速度法的优势在于复杂几何边界处理 (2024 论文核心贡献)。

展望未来，以下方向值得探索：

- **方差缩减**：对涡量法引入重要性采样、控制变量 (参考合作者 Phase 4)；对速度法引入 antithetic variates；

- **GPU 加速:** Biot-Savart 积分天然并行, 适合 GPU 批量加速; WoB 投影可复用 NVIDIA WoSX 框架 [3];
- **混合方法:** 在速度法框架中利用涡量信息增强物理守恒, 或反之;
- **3D 扩展:** 涡量法需处理涡拉伸, 速度法需处理向量势的规范自由度, 两者在 3D 中都需要额外设计。

References

- [1] B. Rioux-Lavoie, R. Sugimoto, T. Owhadi, F. K. Hacene, and T. Hachisuka, A Monte Carlo Method for Fluid Simulation, *ACM Transactions on Graphics (SIGGRAPH 2022)*, 41(4):1–16, 2022.
- [2] R. Sugimoto, C. Batty, and T. Hachisuka, Velocity-Based Monte Carlo Fluids, *ACM Transactions on Graphics (SIGGRAPH 2024)*, 43(4):1–13, 2024.
- [3] R. Sawhney, Walk on Spheres Extensions (WoSX), <https://github.com/nv-tlabs/wosx>, 2025.
- [4] R. Sugimoto, VelMCFluids: Velocity-Based Monte Carlo Fluids, <https://github.com/rsugimoto/VelMCFluids>, 2024.